

AssistiveReviews - Sprint 1 Assignments

Reviewer Profile System & Foundation | Weeks 1-2

Team Development Kickoff | January 26, 2025

Sprint 1 Goal

Implement the complete anonymous reviewer profile system with admin approval workflow

Key concept: Reviewer Unique IDs

1. Each approved reviewer gets a unique ID (used as primary key in database)
2. When logged in, users see their own name on their content
3. When viewing others' content, everyone sees only the anonymous reviewer unique ID (e.g., "Reviewer #12345")
4. Admins see both name and ID

Backend I – User Side

1. Implement reviewer profile creation endpoint
 - Create POST /api/profiles endpoint that accepts user demographic data (name, height, weight, disability type, mobility aids, etc.) and stores it in MongoDB with "pending" status.
 - Generate reviewer unique ID to use as primary key.
2. Build profile read/view endpoints for users
 - Create GET /api/profiles/:id and GET /api/profiles/me endpoints so users can view their own profile and see their approval status.
 - Return both name and unique ID so frontend can display appropriately based on who's logged in.
3. Implement profile update/edit endpoint
 - Create PUT /api/profiles/:id endpoint allowing users to edit their demographic information and profile description.
 - Ensure unique ID never changes.
4. Create database queries for user's own profile management
 - Write MongoDB queries to fetch, filter, and update profile data for the logged-in user.
 - Include queries that return reviewer unique ID for linking to reviews later.

Backend II – Admin Side

1. Implement admin approval/rejection endpoints
 - Create POST `/api/admin/profiles/:id/approve` and `/api/admin/profiles/:id/reject` endpoints that update profile status.
 - Profile unique ID should already exist from creation.
2. Build admin profile review/listing endpoints
 - Create GET `/api/admin/profiles` endpoint that returns all pending profiles with full information (name, demographics, unique ID) for admin review queue.
3. Create admin dashboard data queries (pending profiles, statistics)
 - Write MongoDB aggregation queries to count pending/approved/rejected profiles, show recent activity, and provide admin dashboard metrics.
4. Implement profile status management logic
 - Build the state machine logic for profile statuses: pending → approved/rejected,
5. Implement authorize function in `server/src/middleware/auth.ts`
 - Function to check user roles (admin, reviewer, editor)
 - Required for admin-only routes

Frontend I – User Side

1. Build reviewer profile creation form with all demographic fields
 - Create the multi-step form where users enter name, height, weight, disability type, mobility aids used, profile description, etc. Include validation and clear labels.
2. Implement user's profile view/display page
 - Build the page where users see their own profile information with their actual name displayed.
 - Show approval status and assigned reviewer unique ID (once approved).
 - Display "Pending Approval" message if waiting.
3. Create profile editing interface
 - Build the form that allows users to edit their demographic information and profile description.
 - Pre-populate with existing data, show their name in the header, and handle update submissions.
 - Display unique ID but make it read-only (cannot be edited).
4. Build user profile dashboard

- Create the main profile management page showing user's actual name, profile status, edit button, and notifications (approved/rejected).
- Display reviewer unique ID prominently once approved with explanation: "This is how others will see you on reviews."

Frontend II – Admin Side

1. Build admin dashboard for profile approval queue
 - Create the admin view showing all pending reviewer profiles in a table/card format.
 - Display both the user's actual name AND their unique ID (format: "Jane Doe - Reviewer #12345") so admins can identify profiles.
2. Implement profile approval/rejection interface
 - Build the detail view where admins see full profile information including real name, unique ID, and all demographics.
 - Have clear Approve/Reject buttons with confirmation modals to prevent accidents.
3. Create admin profile management views
 - Build pages for admins to view all profiles (pending, approved, rejected) with filtering and search capabilities.
 - Show both names and unique IDs in listings.
 - Display profile statistics and recent activity.
4. Build admin statistics display
 - Create dashboard widgets showing total profiles, approval rate, pending count, and recent approvals/rejections.
 - Use charts/graphs if time permits.

Full Stack – Integration

1. Coordinate reviewer unique ID generation system across backend/frontend
 - Design and implement the unique ID system (format: sequential numbers, UUIDs, or custom. It's up to you guys).
 - This ID serves as the primary key in the database.
 - Ensure backend generates IDs at profile creation and frontend displays them consistently: user's own name when logged in viewing their content, but unique ID when viewing others' content.
2. Test complete profile creation → approval workflow
 - Manually test the full user journey: create account → fill profile form (including name) → submit → admin sees name + unique ID in queue → admin approves → user sees their name on their profile but knows others see only their unique ID.

3. Handle merge conflicts and integration issues Monitor GitLab merge requests, resolve conflicts between user-side and admin-side code, ensure develop branch stays stable and deployable.
4. Implement profile search/filter across user and admin views
 - Build search functionality allowing admins to filter profiles by name, unique ID, status, date, or demographics. Later enable users to search for reviewers with similar characteristics (showing only unique IDs, not names).
5. Create end-to-end tests for critical paths
 - Write automated tests (or detailed test scripts) for: profile creation with unique ID generation, admin approval, profile editing, and proper display of name vs. unique ID based on logged-in state. Catch integration bugs early.

Sprint 1 Deliverables

1. Functional reviewer profile creation with all demographic fields and unique ID generation
2. Working admin approval system
3. Profile search and filtering for admins
4. Complete user → admin workflow tested end-to-end
5. All endpoints connected to real MongoDB database
6. Proper display logic: users see their own names, but others see unique IDs only

Team Coordination

Integration Syncs

- Backend I + Frontend I sync regularly on user-side features
- Backend II + Frontend II sync regularly on admin-side features

Full Team Sync

- Mid-sprint Teams call (around Week 1.5) for testing.
- Test user flow + admin flow together to catch issues early.

Full Stack Priority

If Full Stack person is handling merges or fixing integration bugs, that takes precedence over their feature tasks. Unblocking the team is #1 priority.

Sprint Demo

End of Sprint 1: Quick demo showing profile creation → admin approval → unique ID

generation working end-to-end. Show the name/ID display logic working correctly. 😊

Reviewer Profile Form Details (subject to change)

REVIEWER PROFILE FIELDS

Account Information (All Users)

Required:

- Full Name (displayed to user only; others see unique ID)
- Email Address (for login and notifications)
- Password (standard authentication)

Reviewer Type (Required - Select One)

This determines which demographic fields appear:

- ☐ Person with Disability
- ☐ Family Member / Friend / Caregiver
- ☐ Medical Professional (Physician, OT, PT, etc.)

FOR: Person with Disability

Required Demographics:

- Height (dropdown: under 4'10", 4'10"-5'2", 5'2"-5'6", 5'6"-5'10", 5'10"-6'2", over 6'2")
- Weight Range (dropdown: under 100 lbs, 100-150 lbs, 150-200 lbs, 200-250 lbs, over 250 lbs)
- Disability Type(s) (checkboxes - select all that apply):
 - Spinal Cord Injury
 - Cerebral Palsy
 - Multiple Sclerosis
 - Muscular Dystrophy
 - Amputation

- Stroke/TBI
- Limited Mobility
- Other (text field)
- Primary Mobility Aid(s) (checkboxes - select all that apply):
 - Manual Wheelchair
 - Power Wheelchair
 - Scooter
 - Walker
 - Crutches/Cane
 - Prosthetic
 - None
 - Other (text field)
- Age Range (dropdown: 18-25, 26-35, 36-45, 46-55, 56-65, 66+)
- Driver Status (dropdown: Drive independently, Drive with hand controls, Drive with other modifications, Passenger only)
- State/Region (dropdown of US states)

Profile Description (Optional):

- Free text field (500 char limit): "Tell others about your experience and what you look for in assistive products"

FOR: Family Member / Friend / Caregiver

Required:

- Relationship to Person with Disability (dropdown):
 - Parent
 - Spouse/Partner
 - Adult Child
 - Sibling
 - Friend

- Professional Caregiver
- Other

About the Person They Care For:

- Person's Disability Type(s) (same checkboxes as above)
- Person's Primary Mobility Aid(s) (same checkboxes as above)
- Person's Height Range (same dropdown as above)
- Person's Weight Range (same dropdown as above)

Profile Description (Optional):

- Free text field (500 char limit): "Describe your role and what perspective you bring to reviews"

FOR: Medical Professional

Required:

- Professional Role (dropdown):
 - Occupational Therapist
 - Physical Therapist
 - Physician (specify specialty in description)
 - Rehabilitation Specialist
 - Mobility Equipment Specialist
 - Other Medical Professional
- Years of Experience (dropdown: <1 year, 1-3 years, 3-5 years, 5-10 years, 10-20 years, 20+ years)
- Areas of Expertise (checkboxes - select all that apply):
 - Wheelchair Seating & Positioning
 - Vehicle Modifications
 - Adaptive Equipment
 - Spinal Cord Injury
 - Stroke/TBI Rehabilitation

- Pediatric Patients
 - Geriatric Patients
 - Other (text field)
- State/Region (dropdown of US states)

Profile Description (Optional):

- Free text field (500 char limit): "Share your professional background and why you're reviewing these products"

Sprint 1 - Team File Helper

Backend I (User-Side)

EDIT:

- server/src/controllers/reviewerController.ts
- server/src/routes/reviewerRoutes.ts

FUNCTIONS TO ADD:

- createReviewerProfile
- getMyReviewerProfile
- updateReviewerProfile
- getReviewerProfileById

Backend II (Admin-Side)

EDIT:

- server/src/controllers/reviewerController.ts
- server/src/routes/reviewerRoutes.ts
- server/src/middleware/auth.ts (add authorize function)

FUNCTIONS TO ADD:

- getPendingReviewerProfiles
- approveReviewerProfile
- rejectReviewerProfile
- getAllReviewerProfiles

Frontend I (User-Side)

EDIT:

- client/src/pages/profile.tsx

CREATE:

- client/src/pages/reviewer-profiles/create.tsx

- client/src/pages/reviewer-profiles/edit/[id].tsx

Frontend II (Admin-Side)

EDIT:

- client/src/pages/admin/index.tsx

CREATE:

- client/src/pages/admin/reviewers/index.tsx
- client/src/pages/admin/reviewers/[id].tsx

Full Stack (Integration)

- server/src/models/ReviewerProfile.ts (add status fields)
- server/src/controllers/reviewerController.ts (add unique ID generation)

ONGOING:

- Review all merge requests
- Resolve merge conflicts
- Test integration